

CAPSTONE PROJECT

Enterprise Quality Engineering with Playwright

Target Application: ParaBank — parabank.parasoft.com
Framework: Playwright v1.53+ | Language: JavaScript (Node.js 20)
Architecture: Page Object Model (POM) | CI/CD: GitHub Actions
Reporting: Allure | Browsers: Chromium · Firefox · WebKit

Services	Test Cases	Browsers
8	139	Chromium · Firefox · WebKit

Repository: `capstoneProject` | Branch: `main`

1. Project Overview

This capstone project engineers a production-grade, enterprise-level test automation ecosystem targeting ParaBank — a fully functional online banking demo application by Parasoft. Built using Playwright v1.53+ with JavaScript (Node.js 20), the framework follows a strict Page Object Model (POM) architecture.

Eight core banking services are automated with a minimum of 15 test cases each (extended to 20 for Bill Payment and Loan Request), delivering 139 automated test cases executed across three browsers: Chromium, Firefox, and WebKit.

The solution integrates a GitHub Actions CI/CD pipeline with a browser-matrix strategy, Allure HTML reporting, and a global database initialization step to ensure consistent test state across all runs.

1.1 Technology Stack

Component	Details
Framework	Playwright v1.53+
Language	JavaScript (Node.js 20)
Browsers	Chromium, Firefox, WebKit

Architecture	Page Object Model (POM) — strict two-layer separation
CI/CD	GitHub Actions — browser matrix (3 parallel jobs)
Reporting	Allure Playwright + allure-commandline, deployed to GitHub Pages
Target Site	parabank.parasoft.com
Test Data	Centralised fixtures/testData.js
DB Init	global-setup.js calls /initializeDB before each run

1.2 Project Goals

- Automate 8 banking services with comprehensive positive, negative, and edge-case coverage
- Execute all tests reliably across Chromium, Firefox, and WebKit
- Maintain a strict Page Object Model — no selector logic in spec files
- Integrate Allure reporting with per-browser artifact upload and merged HTML report
- Deploy merged Allure report to GitHub Pages on every push to main
- Handle ParaBank's AngularJS async rendering with robust waitFor strategies

2. Project Architecture

2.1 Folder Structure

The project lives inside a parabank-project/ monorepo directory with the following layout:

```
parabank-project/
├─ playwright.config.js      # Config — 3 browser projects, Allure reporter
├─ package.json              # Dependencies: @playwright/test, allure-playwright
├─ fixtures/
│   └─ testData.js           # All test data — credentials, payee, loan, profile
├─ pages/
│   └─ BasePage.js           # Base class: this.page + navigate()
│   └─ AuthPage.js           # Login, logout, register
│   └─ AccountPage.js        # Account overview and detail
│   └─ TransferPage.js       # Fund transfer
│   └─ TransactionPage.js    # Transaction history
│   └─ BillPage.js           # Bill payment
│   └─ LoanPage.js           # Loan request
│   └─ ProfilePage.js        # User profile update
├─ tests/
│   └─ global-setup.js       # initializeDB — runs once before all tests
│   └─ auth.spec.js          # 17 test cases
│   └─ account.spec.js       # 16 test cases
│   └─ transfer.spec.js      # 16 test cases
│   └─ transaction.spec.js   # 17 test cases
│   └─ bill.spec.js          # 20 test cases (15 original + 5 new)
```

└─ loan.spec.js	# 20 test cases (15 original + 5 new)
└─ profile.spec.js	# 17 test cases
└─ api.spec.js	# 16 test cases

2.2 Page Object Model Design

Every Page Object class extends `BasePage`, which provides two things only:

- `this.page` — the Playwright Page instance
- `navigate(path)` — calls `this.page.goto(path)`, resolving against `baseUrl`

All locators and action methods are defined in the subclass constructor and methods respectively. No selector string ever appears in a spec file.

```
// pages/BasePage.js
class BasePage {
  constructor(page) { this.page = page; }
  async navigate(path) { await this.page.goto(path); }
}
module.exports = BasePage;
```

2.3 Global Setup — Database Initialisation

ParaBank's public demo server resets its database periodically. The `global-setup.js` file calls the `/initializeDB` endpoint exactly once before any test runs, seeding the `john/demo` account and sample data that every test depends on.

The setup is non-fatal: if the endpoint is unreachable, a warning is logged and the run continues (the DB may already be seeded from a prior run).

3. CI/CD Pipeline — GitHub Actions

3.1 Workflow Design

The GitHub Actions workflow (`.github/workflows/playwright.yml`) implements a two-job strategy:

Job	Responsibility
test	Runs in a 3-way matrix (chromium / firefox / webkit). Each job installs only its browser, runs its project, and uploads browser-specific Allure results as an artifact.
report	Runs after all three test jobs (needs: [test]). Downloads all three allure-results-* artifacts, merges them, generates a single HTML report, uploads it as an artifact, and deploys to GitHub Pages on pushes to main/master.

3.2 Why Only Chromium Was Running Before

The root causes were identified as two separate file-level errors:

1. playwright_config.js (underscore) instead of playwright.config.js (dot) — Playwright could not find the config file and fell back to its internal default, which only runs Chromium.
2. BasePage.js accidentally contained the full TransferPage class code, causing all Page Object classes to crash on import.

Once the config file is renamed and BasePage.js restored to its correct two-method form, the GitHub Actions matrix correctly spawns three parallel jobs and all three browsers run.

3.3 Workflow Triggers

- Push to any branch
- Pull request targeting main or master
- Manual dispatch (workflow_dispatch)

3.4 Key Configuration Choices

Setting	Value & Rationale
fullyParallel	false — ParaBank server cannot handle concurrent sessions reliably
workers	1 — single worker per browser job to avoid account state collisions
retries (CI)	1 — one automatic retry for transient network errors
timeout	120 000 ms — long enough for ParaBank's slow AngularJS pages
actionTimeout	30 000 ms Chromium, 40 000 ms Firefox/WebKit
navigationTimeout	90 000 ms — ParaBank network is slow
headless	true — required for Ubuntu GitHub Actions runners
trace	on-first-retry — captures trace for failures without inflating disk use
screenshot	only-on-failure
video	retain-on-failure

4. Services Under Test

Eight core banking services are covered, each mapped to a dedicated Page Object class and spec file.

#	Service	Page Object	Spec File	Tests
1	Authentication	AuthPage.js	auth.spec.js	17
2	Account Overview	AccountPage.js	account.spec.js	16
3	Fund Transfer	TransferPage.js	transfer.spec.js	16
4	Transaction History	TransactionPage.js	transaction.spec.js	17

5	Bill Payment	BillPage.js	bill.spec.js	20
6	Loan Request	LoanPage.js	loan.spec.js	20
7	User Profile	ProfilePage.js	profile.spec.js	17
8	Internal REST API	N/A (request ctx)	api.spec.js	16

Total: 139 test cases (17 + 16 + 16 + 17 + 20 + 20 + 17 + 16 = 139 across all 8 services)

5. Test Case Breakdown

5.1 Authentication — auth.spec.js (17 Tests)

Covers the full authentication lifecycle: UI verification, valid login, invalid credential handling, logout, and registration page.

ID	Description	Block
TC-AUTH-01	Login panel renders on homepage	Block 1 — Page UI
TC-AUTH-02	Username input is visible	Block 1 — Page UI
TC-AUTH-03	Password input is visible	Block 1 — Page UI
TC-AUTH-04	Log In button is visible	Block 1 — Page UI
TC-AUTH-05	Register link is visible	Block 1 — Page UI
TC-AUTH-06	Valid credentials redirect to account overview	Block 2 — Valid Login
TC-AUTH-07	Logout link appears after successful login	Block 2 — Valid Login
TC-AUTH-08	Account Services heading visible after login	Block 2 — Valid Login
TC-AUTH-09	Wrong password shows error message	Block 3 — Invalid Login
TC-AUTH-10	Wrong username shows error message	Block 3 — Invalid Login
TC-AUTH-11	Empty credentials show error message	Block 3 — Invalid Login
TC-AUTH-12	SQL injection attempt does not log in	Block 3 — Invalid Login
TC-AUTH-13	Logout returns to login page	Block 4 — Logout
TC-AUTH-14	URL contains index or login after logout	Block 4 — Logout
TC-AUTH-15	Register page loads at /register URL	Block 5 — Registration
TC-AUTH-16	First name field visible on register page	Block 5 — Registration
TC-AUTH-17	Register button visible on register page	Block 5 — Registration

5.2 Account Overview — account.spec.js (16 Tests)

ID	Description	Block
TC-ACCT-01	Account overview page loads after login	Block 1 — Page Load
TC-ACCT-02	Page title is visible	Block 1 — Page Load
TC-ACCT-03	Account table is visible	Block 1 — Page Load
TC-ACCT-04	At least one account row is present	Block 1 — Page Load
TC-ACCT-05	Account links visible in table	Block 2 — Account Links
TC-ACCT-06	Clicking first account navigates to activity page	Block 2 — Account Links
TC-ACCT-07	Account detail page shows account number	Block 2 — Account Links
TC-ACCT-08	Account detail page shows balance	Block 2 — Account Links
TC-ACCT-09	Account type is displayed	Block 3 — Account Details
TC-ACCT-10	Available balance is displayed	Block 3 — Account Details
TC-ACCT-11	Accounts Overview link visible after login	Block 4 — Navigation
TC-ACCT-12	Clicking overview link stays on overview page	Block 4 — Navigation
TC-ACCT-13	Back to overview from account detail works	Block 4 — Navigation
TC-ACCT-14	Account table content is not empty	Block 5 — Table Content
TC-ACCT-15	Account table has header row	Block 5 — Table Content
TC-ACCT-16	Multiple accounts can exist in table	Block 5 — Table Content

5.3 Fund Transfer — transfer.spec.js (16 Tests)

ID	Description	Block
TC-TRF-01	Transfer page loads after login	Block 1 — UI
TC-TRF-02	Amount input field is visible	Block 1 — UI
TC-TRF-03	From account dropdown is visible	Block 1 — UI
TC-TRF-04	Transfer button is visible	Block 1 — UI
TC-TRF-05	From account dropdown has at least one account	Block 2 — Dropdowns
TC-TRF-06	To account dropdown is visible	Block 2 — Dropdowns
TC-TRF-07	To account dropdown has at least one account	Block 2 — Dropdowns
TC-TRF-08	Valid transfer shows success title	Block 3 — Valid Transfer

TC-TRF-09	Success page shows transfer complete message	Block 3 — Valid Transfer
TC-TRF-10	Small amount transfer succeeds	Block 3 — Valid Transfer
TC-TRF-11	Decimal amount transfer is accepted	Block 3 — Valid Transfer
TC-TRF-12	Zero amount shows error or stays on page	Block 4 — Invalid Transfer
TC-TRF-13	Empty amount field shows error or stays on page	Block 4 — Invalid Transfer
TC-TRF-14	Page content visible after invalid submit	Block 4 — Invalid Transfer
TC-TRF-15	Transfer link in left nav visible after login	Block 5 — Navigation
TC-TRF-16	Transfer page accessible from left nav	Block 5 — Navigation

5.4 Transaction History — transaction.spec.js (17 Tests)

ID	Description	Block
TC-TXN-01	Account activity page loads after clicking account	Block 1 — Page Load
TC-TXN-02	Right panel content is visible on activity page	Block 1 — Page Load
TC-TXN-03	Page title contains account related text	Block 1 — Page Load
TC-TXN-04	Account overview link visible on activity page	Block 1 — Page Load
TC-TXN-05	Transaction table is visible on activity page	Block 2 — Table
TC-TXN-06	Transaction table has at least one row	Block 2 — Table
TC-TXN-07	Transaction table content is not empty	Block 2 — Table
TC-TXN-08	Transaction table has Date column header	Block 2 — Table
TC-TXN-09	Activity month dropdown is visible	Block 3 — Filters
TC-TXN-10	Transaction type dropdown is visible	Block 3 — Filters
TC-TXN-11	Go button for filter is visible	Block 3 — Filters
TC-TXN-12	Clicking Go button keeps page on activity URL	Block 3 — Filters
TC-TXN-13	Amount search input area visible	Block 4 — Amount Filter
TC-TXN-14	Find Transactions button is visible	Block 4 — Amount Filter
TC-TXN-15	Searching by amount returns page content	Block 4 — Amount Filter
TC-TXN-16	Back to overview from activity page works	Block 5 — Navigation
TC-TXN-17	Activity page accessible from account overview	Block 5 — Navigation

5.5 Bill Payment — bill.spec.js (20 Tests)

Extended from 15 to 20 tests. Block 6 adds five input-validation checks that are pure client-side — no form submission — ensuring reliability across all three browsers.

ID	Description	Block
TC-BILL-01	Bill pay page loads after login	Block 1 — Page Load
TC-BILL-02	Right panel is visible on bill pay page	Block 1 — Page Load
TC-BILL-03	Bill pay page title contains relevant text	Block 1 — Page Load
TC-BILL-04	Payee name input is visible	Block 2 — Form Fields
TC-BILL-05	Address input is visible	Block 2 — Form Fields
TC-BILL-06	Amount input is visible	Block 2 — Form Fields
TC-BILL-07	Account number input is visible	Block 2 — Form Fields
TC-BILL-08	Send Payment button is visible	Block 2 — Form Fields
TC-BILL-09	From account dropdown is visible	Block 3 — From Account
TC-BILL-10	From account dropdown loads accounts	Block 3 — From Account
TC-BILL-11	Submitting valid payment shows response	Block 4 — Payment Submit
TC-BILL-12	Payment response contains payment related text	Block 4 — Payment Submit
TC-BILL-13	Submitting empty form shows page response	Block 4 — Payment Submit
TC-BILL-14	Bill pay link visible in left nav after login	Block 5 — Navigation
TC-BILL-15	Bill pay page accessible via left nav link	Block 5 — Navigation
TC-BILL-16	Payee name input accepts and retains text value	Block 6 — Input Validation
TC-BILL-17	Amount input accepts numeric value	Block 6 — Input Validation
TC-BILL-18	From account dropdown has correct name attribute	Block 6 — Input Validation
TC-BILL-19	Account number input can be cleared and re-entered	Block 6 — Input Validation
TC-BILL-20	Send Payment button has correct input type (submit)	Block 6 — Input Validation

5.6 Loan Request — loan.spec.js (20 Tests)

Extended from 15 to 20 tests. Block 6 mirrors the Bill Payment approach: pure DOM-attribute and fill/read checks with no form submission.

ID	Description	Block
TC-LOAN-01	Loan request page loads after login	Block 1 — Page Load
TC-LOAN-02	Right panel is visible on loan page	Block 1 — Page Load
TC-LOAN-03	Loan page content contains relevant text	Block 1 — Page Load
TC-LOAN-04	Loan amount input is visible	Block 2 — Form Fields
TC-LOAN-05	Down payment input is visible	Block 2 — Form Fields
TC-LOAN-06	From account dropdown is visible	Block 2 — Form Fields
TC-LOAN-07	Apply Now button is visible	Block 2 — Form Fields
TC-LOAN-08	From account dropdown loads options	Block 3 — Account Dropdown
TC-LOAN-09	Loan amount input accepts numeric value	Block 3 — Account Dropdown
TC-LOAN-10	Applying for loan shows response	Block 4 — Loan Application
TC-LOAN-11	Loan response contains loan related text	Block 4 — Loan Application
TC-LOAN-12	Applying with large amount shows response	Block 4 — Loan Application
TC-LOAN-13	Applying with zero amount shows response	Block 4 — Loan Application
TC-LOAN-14	Loan request link visible in left nav	Block 5 — Navigation
TC-LOAN-15	Loan page accessible via left nav link	Block 5 — Navigation
TC-LOAN-16	Down payment input accepts numeric value	Block 6 — Input Validation
TC-LOAN-17	Loan form has all required input fields	Block 6 — Input Validation
TC-LOAN-18	From account dropdown has correct element id	Block 6 — Input Validation
TC-LOAN-19	Loan amount input can be cleared and re-entered	Block 6 — Input Validation
TC-LOAN-20	Apply Now button has correct input type (submit)	Block 6 — Input Validation

5.7 User Profile — profile.spec.js (17 Tests)

ID	Description	Block
TC-PROF-01	Profile page loads after login	Block 1 — Page Load
TC-PROF-02	Right panel is visible on profile page	Block 1 — Page Load
TC-PROF-03	Profile page contains relevant text	Block 1 — Page Load
TC-PROF-04	First name input is visible	Block 2 — Form Fields

TC-PROF-05	Last name input is visible	Block 2 — Form Fields
TC-PROF-06	Address input is visible	Block 2 — Form Fields
TC-PROF-07	City input is visible	Block 2 — Form Fields
TC-PROF-08	Phone input is visible	Block 2 — Form Fields
TC-PROF-09	Update Profile button is visible	Block 2 — Form Fields
TC-PROF-10	First name field is pre-filled by Angular	Block 3 — Pre-filled Data
TC-PROF-11	Last name field is pre-filled by Angular	Block 3 — Pre-filled Data
TC-PROF-12	Updating profile shows response	Block 4 — Update Profile
TC-PROF-13	Profile update response contains success text	Block 4 — Update Profile
TC-PROF-14	Updating with changed data shows response	Block 4 — Update Profile
TC-PROF-15	Profile link visible in left nav after login	Block 5 — Navigation
TC-PROF-16	Profile page accessible via left nav link	Block 5 — Navigation
TC-PROF-17	State and zip fields are present on profile page	Block 5 — Navigation

5.8 Internal REST API — api.spec.js (16 Tests)

Uses Playwright's APIRequestContext (no browser) to test ParaBank's REST endpoints directly, validating JSON structure and HTTP status codes.

ID	Description	Block
TC-API-01	Login endpoint returns 200 for valid credentials	Block 1 — Login API
TC-API-02	Login response contains customer id	Block 1 — Login API
TC-API-03	Login response contains firstName	Block 1 — Login API
TC-API-04	Invalid login returns non-200 status	Block 1 — Login API
TC-API-05	Accounts endpoint returns 200	Block 2 — Customer Accounts
TC-API-06	Accounts response is an array	Block 2 — Customer Accounts
TC-API-07	At least one account is returned	Block 2 — Customer Accounts
TC-API-08	Each account has an id field	Block 2 — Customer Accounts
TC-API-09	Account detail endpoint returns 200	Block 3 — Account Details

TC-API-10	Account detail contains balance field	Block 3 — Account Details
TC-API-11	Account detail contains type field	Block 3 — Account Details
TC-API-12	Invalid account id returns non-200 status	Block 3 — Account Details
TC-API-13	Transactions endpoint returns 200	Block 4 — Transactions
TC-API-14	Transactions response is an array	Block 4 — Transactions
TC-API-15	Each transaction has an id field	Block 4 — Transactions
TC-API-16	Each transaction has an amount field	Block 4 — Transactions

6. Key Technical Challenges & Solutions

ParaBank presents several real-world stability challenges that required careful engineering decisions:

6.1 AngularJS Async Rendering

Challenge: ParaBank's bill payment, loan, transfer, and profile pages are powered by AngularJS. The framework injects form fields into the DOM after `domcontentloaded` fires, so naive assertions fail immediately.

Solution: Every Page Object's `goto` method ends with an explicit `waitFor` on the first form element rather than relying on load-state events.

```
// Example - BillPage.gotoBillPay()
await this.navigate('/parabank/billpay.htm');
await this.page.waitForLoadState('domcontentloaded');
await this.payeeNameInput.waitFor({ state: 'visible', timeout: 35000 });
```

6.2 Transfer: Same-Account Rejection

Challenge: ParaBank silently rejects transfers where `fromAccountId === toAccountId` with no visible error message, causing TC-TRF-08/10 to fail.

Solution: `TransferPage.transfer()` reads all option values from the `toAccountSelect` and programmatically selects the first value that differs from the current `fromAccountSelect` value before clicking Transfer.

6.3 Firefox and WebKit Timeouts

Challenge: Firefox and WebKit fire `domcontentloaded` significantly earlier than Chromium, before any AngularJS bootstrap code has run. This caused all form-field assertions to fail on those browsers.

Solution: `actionTimeout` and `navigationTimeout` are increased to 40 000 ms and 90 000 ms respectively for Firefox and WebKit projects in `playwright.config.js`, and all Page Object `goto` methods use `element-level wait` rather than `network-idle`.

6.4 Flaky TC-TXN-02 (Right Panel Race)

Challenge: After `clickFirstAccount()` + `domcontentloaded`, ParaBank renders `#rightPanel` asynchronously. A `direct toBeVisible()` check raced against the `async` render and failed intermittently.

Solution: Wrapped the assertion in `expect().toPass()` with a 20-second retry loop — the same pattern used by all other panel-content assertions in the file.

6.5 Shared Demo Server State

Challenge: ParaBank's public server is shared by all users worldwide and its database is periodically wiped by Parasoft, removing the `john/demo` account.

Solution: `global-setup.js` calls `GET /parabank/services/bank/initializeDB` before any test runs. The call is non-fatal — a warning is logged if it fails and the run continues.

7. Allure Reporting

7.1 Reporter Configuration

The Allure reporter is registered in `playwright.config.js` alongside the list reporter:

```
reporter: [
  ['list'],
  ['allure-playwright', {
    resultsDir: process.env.ALLURE_RESULTS_DIR || 'allure-results',
  }],
],
```

Each browser job sets `ALLURE_RESULTS_DIR` to `allure-results-{browser}` so results directories never collide.

7.2 Report Generation Flow

3. Each test job uploads its `allure-results-{browser}/` folder as a GitHub Actions artifact (retention: 14 days).
4. The report job downloads all three artifacts and merges them into a single `allure-results/` directory.
5. `allure generate` produces a unified HTML report showing pass/fail breakdown per browser.
6. The report is uploaded as `allure-report` artifact.
7. On push to `main/master`, `peaceiris/actions-gh-pages` deploys the report to GitHub Pages.

7.3 NPM Scripts

Script	Command
npm test	playwright test
npm run test:chromium	playwright test --project=chromium
npm run test:firefox	playwright test --project=firefox
npm run test:webkit	playwright test --project=webkit
npm run allure:generate	allure generate allure-results --clean -o allure-report
npm run allure:open	allure open allure-report
npm run allure:report	generate + open in sequence

8. Evaluation Criteria

Criterion	Requirement	Status
Services	Minimum 8	8 implemented
Test Cases	Minimum 120	139 total
Architecture	Page Object Model	Strict POM enforced
Browsers	Chromium + Firefox + WebKit	All 3 via matrix CI
CI/CD Pipeline	GitHub Actions playwright.yml	Browser matrix configured
HTML Report	Allure reporter	Allure + GitHub Pages
Dual Project	POM + SVG Graphics Engine	Both scaffolded

9. Test Data Reference

All test data is centralised in fixtures/testData.js. No hardcoded strings appear in spec files.

Category	Keys	Used In
validUser	username, password	All spec files — login step
invalidCredentials	wrongPassword, wrongUsername, emptyBoth, sqlInjection	auth.spec.js Block 3
transferData	validAmount, smallAmount, decimalAmount, zeroAmount	transfer.spec.js
payeeData	name, address, city, state, zipCode, phone, account, amount	bill.spec.js

loanData	validAmount, validDownPayment, largeAmount, largeDownPayment, zeroAmount	loan.spec.js
profileData	firstName, lastName, address, city, state, zipCode, phone	profile.spec.js
updatedProfileData	Same keys — alternate values for re-update test	profile.spec.js TC-PROF-14
endpoints	login(), accounts(), account(), transactions() builders	api.spec.js

10. GitHub & Submission

Item	Detail
Repository	capstoneProject
Branch	main
Spec files	tests/ — 8 spec files, one per service
Page Objects	pages/ — 8 page classes + BasePage
Config	playwright.config.js (renamed from playwright_config.js)
CI Workflow	.github/workflows/playwright.yml — matrix: chromium, firefox, webkit
Allure Report	Deployed to GitHub Pages on every push to main